Mobile Application Development

Departamento de Engenharia Informática
Licenciatura em Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa

Master Planner

Final Project Report

Professors:
Ana Duarte Correia
António Teófilo
Pedro Fazenda

Student:
Luís Simões nº 47514

IPL - ISEL - LEIM

June 2026

Contents

List of Figures

1. Executive Summary

Master Planner is an Android mobile application for creating, organizing, and reusing structured routines. The application uses a pirate-inspired visual theme in which users act as captains who chart routes, called **roadmaps**, and fill them with waypoints, called **tasks**. The central goal is to make planning less dry than traditional to-do list applications while still keeping the structure needed for repeated procedures such as study plans, daily routines, bakery workflows, or personal projects.

The problem addressed by the application is the lack of separation, in many simple planning tools, between a high-level plan and the reusable tasks that may appear in several plans. Master Planner separates these concepts explicitly. A roadmap represents the overall route, while tasks represent actionable steps. A global task library, named the Booty Bag, lets users create reusable tasks and import them into roadmaps without rewriting the same information each time.

The target audience is composed of students, productivity enthusiasts, and users who prefer a themed and slightly gamified interface over a neutral productivity application. The application is especially useful for users who build repeated workflows and need to adapt the same task set to different plans.

The main implemented features are **authentication**, **roadmap management**, **task creation**, **task reordering**, a **reusable task library**, **artificial-intelligence-assisted task suggestions**, **simulated freemium restrictions**, **offline support** through Firestore local persistence, **network status feedback**, and a multimedia theme with **custom iconography** and **background music**. The free plan limits users to a small number of roadmaps and tasks, while a simulated Admiral rank removes those limits.

The project was developed in ***Kotlin*** for Android using ***Jetpack Compose*** and ***Stitch.ai*** for the design. **Firebase Authentication** provides account management and email verification. **Firebase Firestore** stores roadmaps, task library items, and roadmap tasks, while snapshot listeners keep the interface synchronized with remote state. Firestore persistent local caching supports offline usage and later synchronization. The **Gemini API** is accessed through **OkHttp** to generate task suggestions from a roadmap title. State is exposed to the UI through ViewModel and StateFlow.

Overall, Master Planner demonstrates a complete mobile application workflow: user login, cloud-backed data management, offline behavior, AI integration, a coherent visual identity, and a reportable architecture that can be explained and extended by a future developer.

2. Application Overview

2.1 Purpose

Master Planner was designed as a thematic routine and task management application. The purpose is to help users transform broad goals into ordered routes. Instead of only storing independent to-do items, the application encourages users to group tasks into roadmaps and to reuse common tasks through the Booty Bag library.

The pirate metaphor is not only decorative. It is used to make the workflow more memorable: roadmaps are routes, tasks are waypoints, the task library is a Booty Bag, premium users have Admiral rank, and offline mode is presented as sailing without signal. This consistent language gives the application a distinct identity and makes routine planning feel less mechanical.

2.2 Main Functionalities

The application includes the following main functionalities:

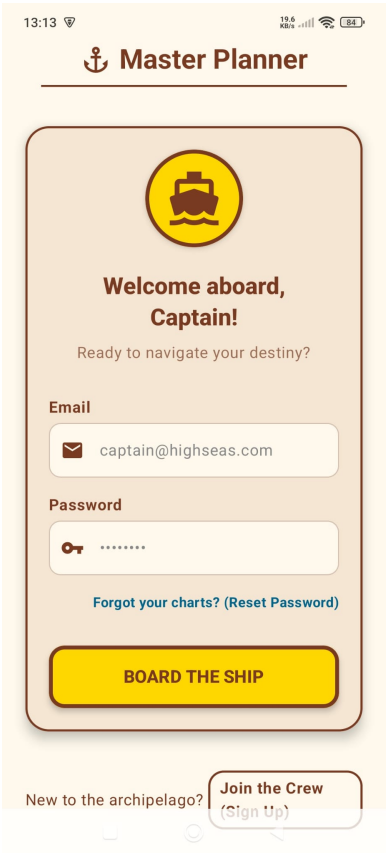
- Account creation and login using Firebase Authentication;
- Email verification before allowing access to the main application;
- Roadmap creation, viewing, deletion, and ordering;
- Task creation and deletion inside a roadmap;
- Drag-and-drop reordering of tasks in the roadmap editor;
- A reusable task library for global tasks;
- Importing library tasks into a roadmap;
- AI suggestions for task names based on the current roadmap title;
- Simulated premium upgrade that removes free-plan limits;
- Offline persistence and a visible network status indicator;
- Custom visual assets and background music.

2.3 Typical User Workflows

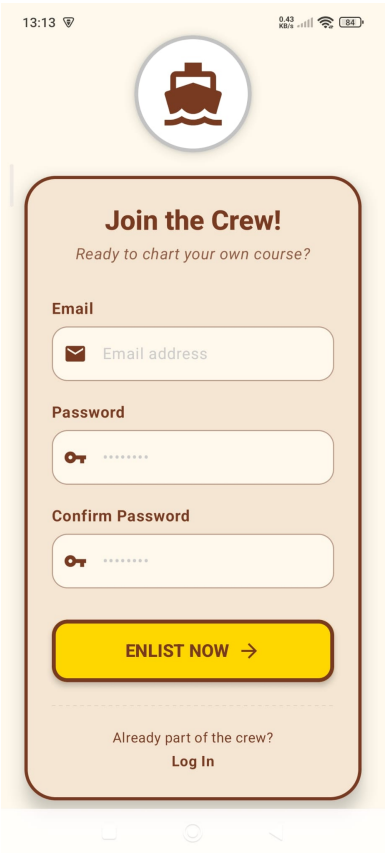
2.3.1 Authentication workflow

The user starts on the splash screen and is redirected either to the login screen or to the main menu, depending on the current Firebase user and email verification state. New users create an account through the sign-up screen and must verify the account by email before logging in.

(a) Splash screen



(b) Login screen

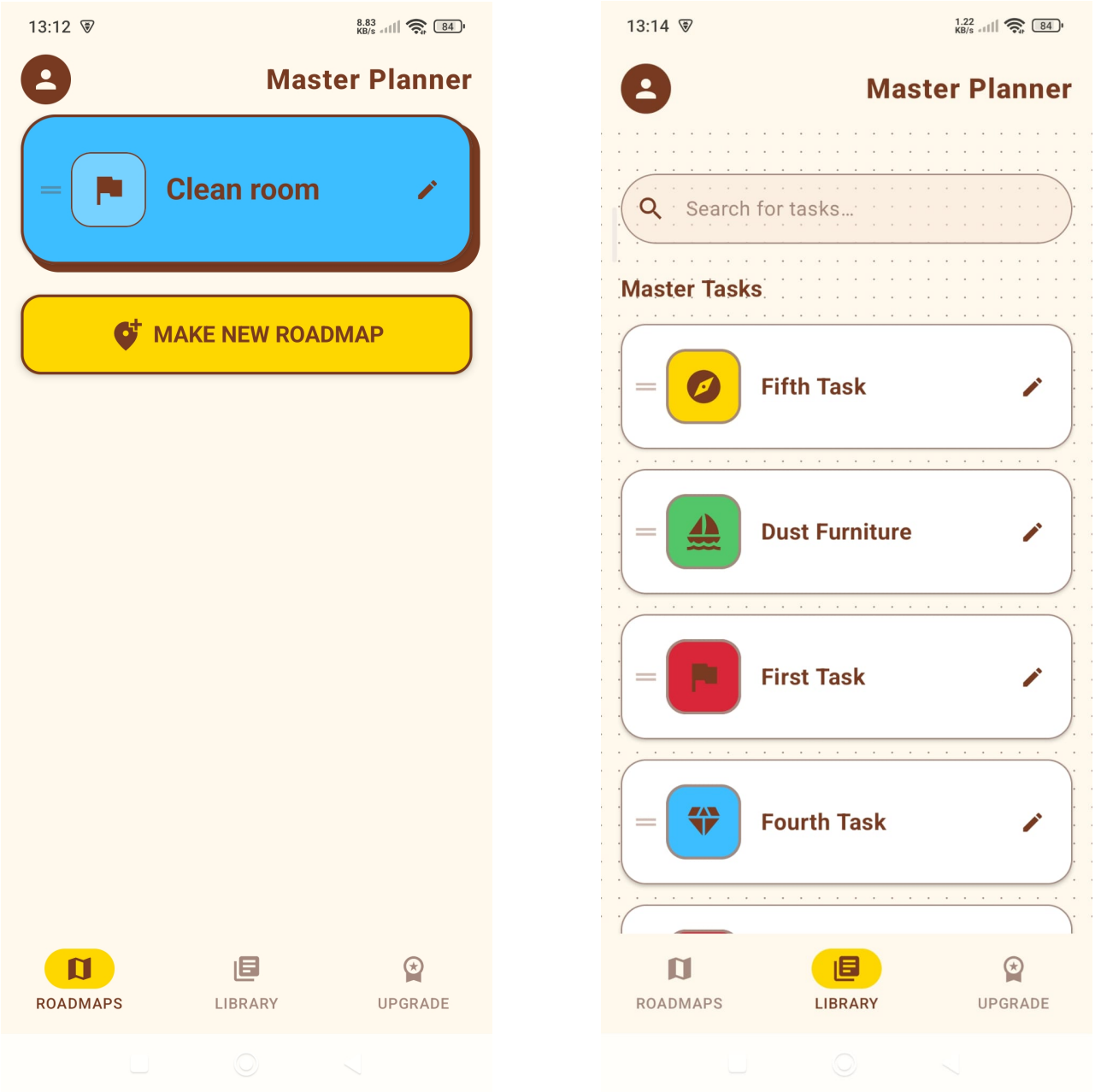


(c) Sign-up screen

Figure 2.1: Authentication and entry screens.

2.3.2 Planning workflow

After login, the user reaches the main menu, where roadmaps are listed. The user can create a new roadmap, open an existing one, manage tasks, or navigate to the Booty Bag task library. Inside the roadmap editor, tasks can be created, deleted, and reordered to define the sequence of the route.



(a) Main menu

(b) Task library

Figure 2.2: Main application screens.

3. Architecture and Technologies

3.1 Technology Stack

Technology	Role in the application
Kotlin	Main programming language for Android implementation.
Jetpack Compose	Declarative user interface framework used for all screens.
Stitch.ai	UI component system and visual styling foundation.
ViewModel	Holds screen-independent state and coordinates data operations.
StateFlow	Exposes observable state from the ViewModel to Compose screens.
Firebase Authentication	Handles login, account creation, and email verification.
Firebase Firestore	Cloud database for roadmaps, tasks, and library entries.
Firestore local persistence	Provides offline cache and automatic synchronization.
OkHttp	Performs HTTP calls to the Gemini API.
Gemini API	Generates AI-assisted task suggestions.
SharedPreferences	Stores simulated premium state through the PremiumManager.
sh.calvin.reorderable	Provides drag-and-drop reordering support in Compose lists.
Android media resources	Provide background music and custom imagery.

Table 3.1: Technologies used in Master Planner.

3.2 Simplified Architecture

The application follows an MVVM-style structure. The Activity owns navigation state and initializes application services such as Firestore persistence, background music, and the premium manager. The UI layer is composed of independent Compose screens. The ViewModel exposes StateFlow values and centralizes operations that affect roadmaps, tasks, AI suggestions, and network status. The data layer contains Firebase references, data models, the Gemini service, network monitoring, and premium state management.

3.3 Data Model

The application uses three core data concepts. A user owns a set of roadmaps. Each roadmap contains a subcollection of tasks. A separate task library stores reusable tasks. Roadmap tasks are copied into

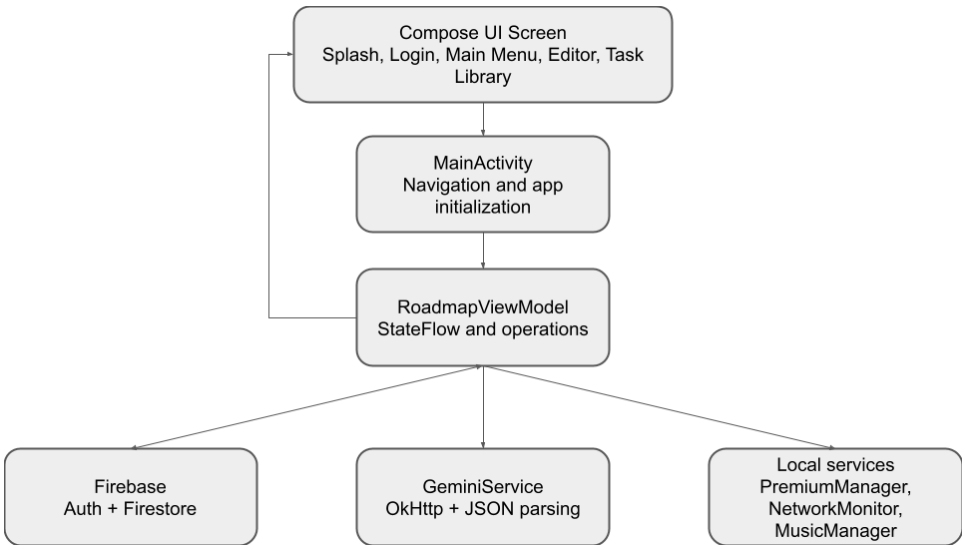


Figure 3.1: Simplified architecture of the application

individual roadmaps so that deleting or editing a library entry does not necessarily break existing roadmaps.

Entity	Main responsibility
Roadmap	Represents a named route or plan, including an identifier, title, ordering information, and timestamp.
Task	Represents an actionable waypoint with title, duration or metadata, icon, color, and timestamp.
RoadmapItemEntry	Represents the relationship between a roadmap and its ordered entries when needed by UI operations.

Table 3.2: Core application data model.

3.4 Firebase Structure

The intended Firestore structure is user-scoped. Roadmaps are stored under the authenticated user's UID, and each roadmap has its own task subcollection. This supports ownership rules and avoids exposing one user's data to another user. The library collection is also user-scoped, which allows each user to maintain a personal Booty Bag.

Listing 3.1: Conceptual Firestore structure.

```
users/{uid}/roadmaps/{roadmapId}
users/{uid}/roadmaps/{roadmapId}/tasks/{taskId}
users/{uid}/task_library/{taskId}
```

3.5 Navigation

Navigation is implemented with an enum-based screen state in MainActivity. This approach is simple and appropriate for a small academic project with a limited number of screens. The current screen, current roadmap identifier, current roadmap title, and previous screen are stored as Compose state. Although a full Navigation Compose graph would scale better for larger applications, the selected approach is easy to inspect during oral discussion and avoids unnecessary complexity.

4. Main Features

4.1 Authentication and Email Verification

Firebase Authentication is used to create accounts and sign users in with email and password. The application checks email format, password confirmation, and email verification. After a successful sign-up, users receive a verification email. The login flow refuses access when the email has not been verified, which improves account integrity and makes the authentication state explicit.

4.2 Roadmap Management

The Main Menu displays the user's roadmaps and allows roadmap creation and deletion. Roadmaps are retrieved through Firestore snapshot listeners, which means the UI is updated whenever remote data changes. The list is sorted using the stored sort order and timestamp, providing stable presentation of user-created plans.

Listing 4.1: Listening to roadmap updates with Firestore and StateFlow.

```
fun listenToRoadmaps() {
    Utility.collectionReferenceForRoadmaps
        .addSnapshotListener { snapshot, error ->
            if (error != null || snapshot == null) return@addSnapshotListener
            val list = snapshot.documents
                .mapNotNull { doc -> doc.toObject<Roadmap>()?.copy(id = doc.id) }
                .sortedWith(compareBy({ it.sortOrder }, { -(it.timestamp?.seconds ?: 0L) }))
            _roadmaps.value = list
        }
}
```

4.3 Roadmap Editor and Task Ordering

The Roadmap Editor shows the tasks inside a selected roadmap. When the user opens a roadmap, the ViewModel attaches listeners to both the roadmap document and its tasks subcollection. Existing listeners are removed when switching to another roadmap to prevent stale updates. Task ordering is supported through a reorderable Compose list, allowing the user to define the chronological order of the routine.

4.4 The Booty Bag Task Library

The Booty Bag is a reusable task library. It allows the user to create general tasks independently from any single roadmap. These tasks can later be imported into a roadmap. This design avoids repeated typing and supports the application's distinction between a general task template and a roadmap-specific task instance.

4.5 AI-Assisted Suggestions

The Discovery Compass feature uses the Gemini API to suggest exactly six short task names based on the current roadmap title. The prompt requests a JSON array only, making the response easier to parse and transform into UI suggestions. The implementation uses OkHttp, runs on an IO dispatcher, and returns an empty list when the response fails or cannot be parsed.

Listing 4.2: Prompt used for AI task suggestions.

```
val prompt = """
    You are helping a user plan tasks for their roadmap titled: "$roadmapTitle".
    Suggest exactly 6 short, actionable task names that would fit this roadmap.
    Respond ONLY with a JSON array of strings. No explanations, no markdown, no extra text.
""".trimIndent()
```

The most important challenge in this feature was controlling the shape of generated output. AI models may add Markdown, explanations, or invalid JSON if the prompt is too loose. The final design therefore uses a narrow prompt and validation through JSON parsing. Invalid output is treated as failure rather than being shown directly to the user.

4.6 Freemium Simulation

The application simulates a freemium model. Free users are limited in the number of roadmaps and tasks they can create. The Freemium screen presents the Admiral rank and a simulated upgrade flow. The premium state is stored locally through PremiumManager. This satisfies the freemium requirement without implementing real payments, which would be unnecessary and risky for an academic project.

4.7 Offline Support and Network Feedback

Firestore persistent local caching is enabled during application startup. This allows the user to continue seeing and editing cached data while offline, and Firestore later synchronizes changes when connectivity is restored. The network monitor exposes online status to the ViewModel so the UI can show a themed offline banner when the device is sailing without signal.

Listing 4.3: Firestore persistent local cache initialization.

```
val settings = FirebaseFirestoreSettings.Builder()
    .setLocalCacheSettings(
        PersistentCacheSettings.newBuilder()
            .setSizeBytes(50L * 1024 * 1024)
            .build()
    )
    .build()
FirebaseFirestore.getInstance().firestoreSettings = settings
```

4.8 Multimedia and Thematic Identity

The project includes a custom treasure map icon, launcher assets, pirate-themed visual language, and a background music resource. These elements support the multimedia requirement and reinforce the application's identity. The design uses terms such as Roadmap, Booty Bag, Admiral, and Sailing without Signal consistently across documentation and interface text.

5. Use of Artificial Intelligence

5.1 AI Tools Employed

AI was used in two distinct ways. First, the application itself integrates Gemini as a runtime feature for generating task suggestions. Second, AI tools were used during development as assistants for architecture discussion, debugging, documentation drafting, prompt refinement, and code review.

5.2 Tasks Supported by AI

AI assistance supported boilerplate generation, explanation of Android and Compose concepts, architectural comparison, debugging of compiler errors, prompt design for Gemini, and drafting of documentation. The prompts folder records specific examples for architecture, Compose UI, debugging, and refactoring.

5.3 Relevant Prompt Examples

Examples of useful development prompts include asking for a clean architecture proposal for a Compose application using Firebase, asking for a Material 3 screen layout for a task library, and asking for help interpreting compiler errors related to the reorderable list library. The runtime prompt is the one shown in the previous chapter: it asks Gemini to generate exactly six short actionable task names as JSON.

5.4 Where AI Was Helpful

AI was most helpful for accelerating learning and reducing repetitive work. It helped generate initial structures, identify likely causes of compiler errors, suggest UI improvements, and explain unfamiliar APIs. It was also useful for improving the clarity of documentation and for turning rough ideas into structured requirements and user stories.

5.5 Where AI Was Incorrect or Insufficient

AI produced several incorrect or insufficient suggestions during the project. Examples included invalid Compose padding overloads, outdated assumptions about the reorderable library API, incorrect shell commands for the operating system, and premature diagnosis of Firestore bugs before Logcat evidence was available. These mistakes showed that AI output must be treated as a hypothesis, not as a guaranteed solution.

5.6 Validation Methods

AI-generated outputs were validated through compilation, manual testing, Logcat inspection, comparison with installed library documentation, and consistency checks against the project architecture. For the Gemini feature, validation is also performed at runtime by parsing the response as JSON. If the response is invalid or the request fails, the application returns an empty suggestion list instead of showing unreliable content.

6. Application Quality

6.1 Testing Strategy

The testing strategy combines manual and automated checks. Manual testing was especially important for authentication, Firestore synchronization, offline behavior, task creation, task deletion, drag-and-drop reordering, and the freemium flow. The project includes default unit and instrumented test files, but more extensive automated coverage remains future work.

The most important scenarios to test are:

- Creating an account and verifying email before login;
- Creating, opening, and deleting roadmaps;
- Creating tasks in a roadmap and in the Booty Bag;
- Importing reusable tasks into roadmaps;
- Reordering tasks and confirming order persistence;
- Triggering AI suggestions and handling failure cases;
- Reaching free-plan limits and simulating upgrade;
- Editing data while offline and confirming later synchronization.

6.2 Error Handling

The application handles many operations with success and failure callbacks. Firestore write failures return false or null to the caller, and AI failures return an empty list. Authentication validation checks email format, password presence, and password confirmation before sending requests to Firebase. Toast messages are used for user-facing feedback in several flows.

6.3 Usability Considerations

The interface uses a consistent theme, recognizable icons, and clear screen separation. The main flows are short: login, select roadmap, edit tasks, and return to the main menu. Empty states and limits are represented by explicit UI states. The pirate theme is used to make the experience more engaging, but the underlying actions remain understandable.

6.4 Security and Privacy

Authentication is delegated to Firebase Authentication, so passwords are not stored in Firestore. Firestore paths are user-scoped through the authenticated UID, which supports ownership boundaries. A relevant limitation is that the current Gemini API key appears in source code. In a production project, this should be moved to a secure backend or protected configuration because mobile application secrets can be extracted from distributed apps.

6.5 Known Limitations

The current implementation has several limitations. The freemium system is simulated locally and is not a real billing solution. Automated tests are limited. The Gemini integration depends on network availability and on valid model output. The navigation approach is simple and does not use a full Navigation Compose back stack. Advanced collaboration, calendar integration, analytics, and cross-user sharing are outside the current scope.

7. Lessons Learned

This project reinforced several technical and non-technical lessons. Technically, it showed the importance of separating UI state from data operations. Compose is powerful, but it requires disciplined state management to avoid unnecessary recomposition and confusing data flow. Firestore snapshot listeners simplify real-time updates but require careful listener lifecycle management. AI integration requires strong validation because generated responses are not deterministic.

The project also improved understanding of Firebase Authentication, Firestore data modeling, offline persistence, Kotlin coroutines, StateFlow, and Compose UI development. Debugging skills improved through Logcat analysis, compiler error interpretation, and library API verification.

Non-technically, the project showed the value of maintaining documentation during development. The documentation files helped preserve project vision, requirements, architecture decisions, user stories, and AI usage history. This made it easier to write the final report and to prepare for oral discussion.

If the project were restarted, more time would be allocated to automated testing and to separating concerns even further. A Navigation Compose graph would be introduced earlier, AI keys would never be stored in the client, and a clearer repository abstraction would be used between the ViewModel and Firebase operations.

8. Future Work

Future work can be divided into feature improvements, technical improvements, user interface improvements, and integrations.

8.1 New Features

Potential new features include collaborative roadmaps, public templates, calendar export, reminders, recurring roadmaps, progress analytics, notifications, and shared task packs. The Booty Bag could be expanded into a searchable template marketplace.

8.2 Technical Enhancements

The project could be improved by adding a repository layer, introducing Navigation Compose, increasing automated test coverage, adding continuous integration, and moving Gemini calls to a backend service. Security rules should be documented and tested more thoroughly. The premium system should be server-validated if real payments are ever introduced.

8.3 User Interface Improvements

The UI could be improved with richer empty states, onboarding, better accessibility testing, tablet layouts, animation polish, and clearer progress indicators for AI suggestions and offline synchronization.

8.4 Additional Integrations

Useful integrations include Google Calendar, Android notifications, cloud functions for secure AI calls, and optional export to PDF or Markdown for completed roadmaps.

9. Conclusion

Master Planner delivers a functional Android planning application with a coherent theme and a meaningful set of mobile development features. It includes authentication, cloud-backed state, offline support, reusable task management, AI-assisted suggestions, multimedia assets, and a simulated freemium model. The project demonstrates both product thinking and technical implementation.

The greatest value of the application is its structured approach to planning. By separating roadmaps from reusable tasks, it supports repeated workflows more effectively than a flat to-do list. The pirate-inspired identity adds personality without hiding the practical purpose of the tool.

From a learning perspective, the project provided experience with modern Android development, Firebase, Compose state management, AI integration, and responsible use of AI tools. The final result is not only an application, but also a documented development process with clear limitations and realistic future improvements.

A. Appendix: Requirements Traceability

Requirement	Implementation evidence
3–5 screens	Splash, Login, Sign Up, Main Menu, Roadmap Editor, Create Task, Task Library, Freemium.
Shared state	Firebase Firestore stores user roadmaps and tasks and updates UI through snapshot listeners.
AI	Gemini API provides task suggestions from roadmap titles.
Multimedia	Custom iconography, treasure map logo, launcher assets, and background music.
Freemium	Free limits and simulated Admiral rank through PremiumManager.
Offline	Firestore persistent local cache and network status monitoring.

Table A.1: Traceability between DAM requirements and project implementation.

B. Appendix: Demonstration Checklist

- Android Studio project opens and Gradle sync completes.
- Firebase project configuration is available through google-services.json.
- Test account is created and email verified.
- Physical device or emulator is prepared before discussion.
- Application is installed and login credentials are available.
- Network can be disabled and re-enabled to demonstrate offline behavior.
- Gemini suggestion feature is tested before the presentation.
- The final report is stored in the repository root as FinalProjectReport.pdf.

C. Appendix: Selected Project Files

- `MainActivity.kt`: application initialization and screen navigation.
- `RoadmapViewModel.kt`: roadmap, task, library, AI, and network state management.
- `GeminiService.kt`: OkHttp-based Gemini API integration.
- `PremiumManager.kt`: simulated premium state.
- `NetworkMonitor.kt`: connectivity monitoring.
- `MainMenuScreen.kt`: roadmap list and main actions.
- `RoadMapEditorScreen.kt`: roadmap task management.
- `TaskLibraryScreen.kt`: reusable task library.
- `FreemiumScreen.kt`: Admiral rank upgrade screen.